

Mnemon: Raw Records, Fast Judgments, Slow Thoughts

Guangren Wang

Abstract

Long-term memory lets an LLM assistant use a history it can no longer reread, and most memory systems build it by rewriting conversations into facts, graphs or typed memories at write time. We argue that the work of memory divides, as thinking does, into two systems. Most of it is fast System 1 work: many small, independent yes/no judgments about records, such as whether a record is needed or no longer current, which a decision model makes by the dozen in a third of a second. Only a little is slow System 2 work: writing a few search queries, naming what the reply needs and composing the answer, which an LLM does well but slowly. We present Mnemon, a memory agent built on this division. It keeps conversations as raw, dated records; an LLM (System 2) plans searches over them, a decision model, Jev (System 1), judges what the searches return, and rules with explicit budgets turn the judgments into a small View for an unchanged answering model. A background pass consolidates each record once into topic timelines, value histories and standing instructions linked to the records, so that questions about a whole conversation reach evidence their own searches miss. Because nothing is decided about a record when it is written, the same agent can read any store that returns dated records.

With gpt-4.1-mini answering, as in a public re-evaluation of 14 systems, Mnemon scores 91.7% on LoCoMo, the highest among them, and 83.8% on LongMemEval-S, from under 4k tokens of context per question, with the lowest effective cost index on LoCoMo. With a reasoning model answering, it reaches 92.2% on LoCoMo and 94.4% on LongMemEval-S, the latter on par with the best published results. From 100K to 10M tokens of history on BEAM, its cost per question grows by a factor of 1.11. On the same records, Jev separates gold evidence better than two LLMs and is 3–11 times faster.

1 Introduction

An assistant that talks with the same person for months accumulates more history than it can reread at every turn. Rereading everything grows more expensive with every conversation and stops being possible once the history outgrows the context window. Long-term memory systems therefore decide which parts of the history the assistant sees, and most of them decide it when the history is written: Mem0 extracts and updates facts [7], Zep builds a temporal knowledge graph [40], and MemOS, EverMemOS, Nemori and MIRIX organize conversations into memory units, episodes or typed stores [16, 25, 34, 53].

The cost of write-time extraction. Rewriting a conversation at write time pays for understanding before the question is known. Every message is processed whether or not it is ever asked about, and what the extractor drops or distorts cannot be recovered when a question finally reveals what mattered. It also moves judgment away from the moment with the most information for it: the question, the recent dialogue and the candidate records are known together only when the question is asked. And it ties memory to a schema: the extractor decides in advance what counts as a fact, an entity or a preference, so each new kind of data, such as documents, tasks or logs, needs a new extraction schema.

Fast judgments, slow thoughts. Dual-process accounts of thinking separate a fast, automatic System 1 from a slow, deliberate System 2 [18]. The work of memory divides the same way. Most of it is System 1 work: small, independent yes/no judgments with explicit criteria, such as whether the reply should use this record, whether it is no longer current or whether it gives the second of the two dates the question needs. Decision models such as Jev make such judgments by the dozen in a third of a second [1]. Only a little is System 2 work: writing a few search queries, naming what the reply needs and composing the answer from what is shown, which an LLM does well but slowly. Because the judging is fast, Mnemon can read raw, dated records when a question arrives instead of rewriting them in advance: an LLM plans (System 2), Jev judges (System 1), and rules with explicit budgets decide which searches to

page or rewrite and what to show. Slow work that no reply can wait for runs in the background: for evidence that no question points to, such as an instruction given once, every mention of a topic or a value that changed, Mnemon consolidates each record once into an index of topic timelines, value histories and standing instructions that links back to the records. The index directs reading, and the records remain the evidence.

Memory without a write-time schema. Deferring interpretation to read time also frees memory from the structure of its store. Nothing about a record is decided when it is written, so Mnemon needs from a store only a search route that returns dated records: a conversation journal, a collection of documents or a task list can feed the same agent without a new extractor. The consolidated index sits on top of the records and points back to them, so it never becomes the store’s schema. Memory of this kind can be added wherever records can be searched, which makes it a general way to give agents a past beyond conversation; we evaluate it on conversational memory, the setting with public benchmarks.

Our contributions. Mnemon runs as a *replica*, a second instance of an open-source agent harness [11], and hands the unchanged main agent one View per turn. Our contributions are:

- **Memory as System 1 and System 2** (Sections 3 to 5): a memory agent that gives judging to a decision model and planning and answering to an LLM, connected by rules under explicit budgets that use only the order and the yes/no of judgments.
- **Memory without a write-time schema** (Sections 3, 4.1 and 6.6): raw records about which nothing is decided until a question arrives, which no write-time processing can improve on in information (Equation 1), and a consolidated index that points back to them, so that the agent reads any store that returns dated records. Under one protocol, Mnemon is 7.3 points more accurate than a concurrent system that uses the same decision model to organize memory at write time.
- **Accuracy at small context** (Section 6.2): compared with the 14 memory systems that OmniMemEval re-evaluated with gpt-4.1-mini answering [33], Mnemon is the most accurate on LoCoMo (91.7%) and second on LongMemEval-S (83.8%), from under 4k tokens of context per question, with the lowest effective cost index on LoCoMo.
- **A stronger System 2, bounded cost** (Sections 6.3 and 6.4): with a reasoning model as System 2, Mnemon reaches 92.2% on LoCoMo and 94.4% on LongMemEval-S, on par with the best published results on LongMemEval-S; from BEAM-100K to BEAM-10M, with 80 times as many records, its cost per question grows by a factor of 1.11.
- **Judging belongs to System 1** (Section 6.5): on the same records, Jev separates gold evidence better than two LLMs (AUC 0.942 against 0.900 and 0.853) and is 3–11 times faster.

Figure 1 summarizes the comparison on the two most widely reported benchmarks: Mnemon is the only system above 80% on both that sends the answering model fewer than 4k tokens per question.

2 Related Work

Memory for LLM agents. MemGPT, now Letta, pages information in and out of the context window [22, 39]. Most later systems work at write time: they extract facts [7], build temporal knowledge graphs [40] or linked notes [56], organize memory into tiers, cubes or typed stores [19, 25, 53], consolidate conversations into episodes [16, 34], or compress and restructure what they extract [4, 13, 20, 21, 24, 26, 28, 45, 59, 61, 63]. MemMachine keeps raw episodes indexed by sentence [52], and SmartSearch reranks raw history [12]. Mnemon keeps raw records as the only evidence, reasons at read time, and consolidates only into an index that points back to the records.

Retrieval and judgment. Mnemon retrieves with BM25 [41] and dense embeddings [37], fused by reciprocal rank fusion (RRF) [8], and with hypothetical-document queries [15], as in retrieval-augmented generation [23]. Jev acts as a reranker [36, 42] whose judgments answer explicit propositions and drive actions, as in ReAct and Self-RAG [2, 57].

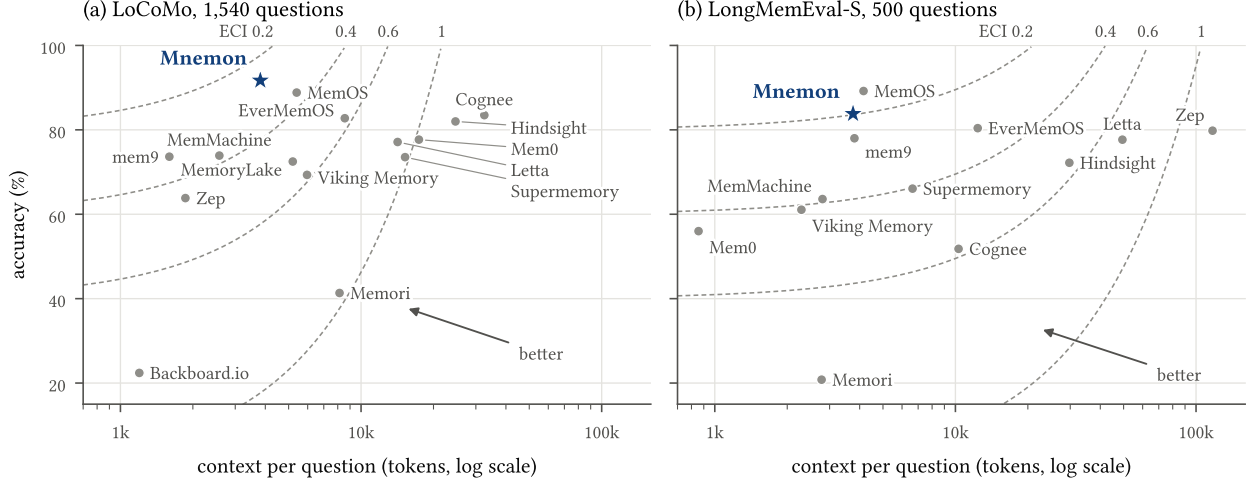


Figure 1: Accuracy against context per question for Mnemon (star) and the 14 systems re-evaluated by OmniMemEval, all with gpt-4.1-mini answering. Dashed lines join points of equal effective cost index (Equation 2); up and to the left is better.

Fast and slow thinking. Dual-process accounts separate fast, automatic judgment from slow deliberation [18], and AI systems have borrowed the division to pair fast and slow components [3], for instance a small, fast action model with an LLM planner in interactive agents [27]; LLM cascades route easy queries to cheap models and hard ones to expensive models [6]. Memory systems have recently adopted the division as well: D-Mem falls back from vector retrieval to exhaustive LLM reading [58], DCPM and Engram pair a fast write path with slow consolidation into schemas or a bi-temporal knowledge graph [14, 51], and Jev-Mem uses Jev to type and link each turn at write time and to steer retrieval over the resulting graph [17]. Mnemon applies the division on the read path instead: System 1 judges raw records once a question is known, System 2 plans the searches and composes the answer, and nothing is decided when a record is written.

Evaluating memory. LoCoMo [29] and LongMemEval-S [55] are the standard benchmarks for long conversations; BEAM reaches 10M tokens [44], and HaluMem measures hallucination [5]. Protocols and LLM graders [62] vary between papers, and several self-reported scores fell by 10–40 points when OmniMemEval re-evaluated 14 systems under one protocol [33]. We therefore follow that protocol and list published claims separately.

3 Problem Setting

We consider an assistant, the *main agent*, that converses with one user over many sessions. Its *history* is a sequence of sessions, each a dated sequence of messages. At each user turn a *memory agent* may read the history, and it publishes a *View*: a bounded block of context that the main agent receives with the message before it replies. The main agent is otherwise unchanged, and the memory agent never edits its reply. In the benchmarks we use, a question is asked in a fresh session after its history has been written, so the answer depends on the View alone.

Formal model. We write the history as a sequence of dated records $H = (r_1, \dots, r_n)$, each a timestamp and a short span of dialogue. At a user turn, let M be the recent dialogue, ending with the user’s message, and let Y be what the reply should convey; Y depends on the message, which is not known when the history is written. A memory agent maps M and H to a View $V = F(M, H)$ within a budget B on its length, and the main agent replies from M and V . A *write-time* memory first applies a transformation ϕ fixed before any question, such as extraction into facts or a graph, and reads only its output: $V = F(M, \phi(H))$. Since $\phi(H)$ is a function of H , the variables form a Markov chain $Y \rightarrow (M, H) \rightarrow (M, \phi(H))$, and the data processing inequality [9] gives

$$I(Y; M, \phi(H)) \leq I(Y; M, H), \quad (1)$$

with equality only if $\phi(H)$ retains all that H says about the answer to every question that may be asked. Processing at write time can therefore lose information about questions not yet asked, and it cannot add any. Equation 1 does

not make raw records sufficient: the View must still fit B , and choosing it is the read-time work that Mnemon divides between two systems. An index $X = \psi(H)$ kept beside the records, rather than in their place, loses nothing, since $I(Y; M, H, X) = I(Y; M, H)$; it can only make the relevant records easier to find.

Two kinds of work. Following dual-process accounts of thinking [18], we divide the read-time work into two kinds. *System 1 work* consists of *judgments*. A judgment evaluates a yes/no proposition π with an explicit criterion about one item x , a record or an index item, in a shared state c , such as the recent dialogue and the needs under consideration, and returns the probability $p_\pi(x | c) \in [0, 1]$ that π holds: does the reply need this record, is it no longer current, does it satisfy this need. Judgments on one state that take no other judgment’s result as input form a *wave*, which a decision model answers in parallel batches, dozens of judgments in a fraction of a second; the time a turn spends on System 1 work therefore grows with its number of waves, not of judgments. *System 2 work* is open-ended generation $g(c)$ that does not decompose into such propositions: writing queries for records not yet seen, naming what a reply needs, and composing an answer that counts, compares dates or chains facts. It suits an LLM, which does it well but slowly, so a memory agent should keep as little of it on the read path as it can and move the rest to the background. We borrow the division of labor, not the view that fast judgment is error-prone: our judgments have explicit criteria, and a model built for them makes them well (Section 6.5).

Harness. We build the memory agent in DeepSeek Harness (DSH), an open-source agent harness in which every capability is a plugin [11]. Its dsh-mnemon suite provides memory through *Sources*, which expose stores such as a journal of records through read and search routes, and *Strategies*, which decide what the harness publishes into the agent’s context as one View per turn [50]. Mnemon is agnostic to the structure of a store: it needs from a Source only a search route that returns dated records, so a store of documents, tasks or past sessions can feed the same View as the conversation journal. In this paper all memory lives in one Source, the journal.

Cost measure. A memory system incurs cost at three points: when it processes the history (write time), when it decides what to show (read time) and when the answering model reads what it was shown (answer time). Only the last is reported for every system in public re-evaluations, as the number of tokens of context sent to the answering model per question [33]. Accuracy per unit of cost rewards systems that are cheap and often wrong, so we price errors instead. If a wrong answer is repaired by one full-context answer of cost c_{full} , a system with accuracy a that sends c tokens of context per question has the expected cost

$$\text{ECI} = (1 - a) + \frac{c}{c_{\text{full}}} \quad (2)$$

in units of c_{full} , which we call its *effective cost index*. Lower is better, and answering from the full history costs at least one. Because read-time and write-time costs are not published for other systems, we compare costs across systems by this index alone and report our other costs separately.

4 Design

At every user turn Mnemon decides what the main agent should see before it replies (Figure 2). It divides the work as Section 3 does: an LLM plans the searches and names what the reply needs (System 2), Jev judges the pooled records (System 1), rules with explicit budgets connect the two, and slower System 2 work, consolidating the history into an index, runs in the background. Sections 4.1 to 4.3 describe the memory, the two systems that read it and the rules; Algorithm 1 summarizes one turn.

4.1 Memory: Raw Records and a Consolidated Index

Records. Each session is written as it happened, as short `speaker : text` chunks, each headed by the session’s number and date. No LLM is on the write path of a record, and nothing about it is decided when it is written; the only computation is one embedding, so the write path is the same whatever the records are about.

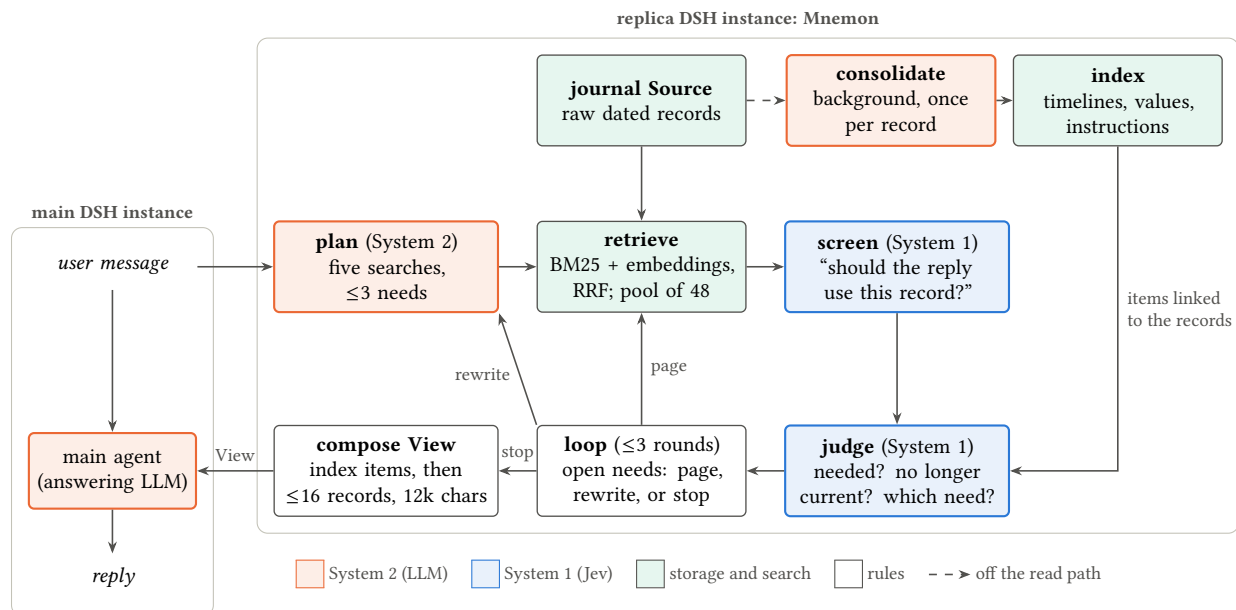


Figure 2: Mnemon at one user turn. System 2, an LLM, plans the searches; System 1, Jev, screens and judges the raw records they return and the index items those records link to; rules loop on needs that are still open and publish a View into the main instance, whose LLM answers. Off the read path, System 2 consolidates each record once into an index that points back to the records.

Consolidation. Questions about a whole conversation, such as every mention of a topic, the current value of something that changed or an instruction given once, need evidence that the question’s own searches rarely find. Gathering it is System 2 work too slow for the read path, so a background job reads the journal once, in order, and a small LLM folds it, batch by batch, into an *index* of three kinds of items, each linked to the records it came from: events on topic timelines; dated histories of values that can change, marked where a value contradicts an earlier one; and standing instructions and preferences. Items point to records and never replace them: the index is an overlay on the records, not their schema, and Mnemon reads the records with or without it.

4.2 Reading with Two Systems

Plan (System 2). Planning is System 2 work: it must anticipate how the records that answer a question are worded and what the reply will have to say. Two LLM calls read the last six messages in parallel. The *search plan* writes three searches in the words a stored record would use, plus one hypothetical record that would answer the message (HyDE [15]); with the message itself, a turn starts with five searches. The *need plan* lists at most three information needs, marked NEED, or ALL when the reply needs every instance of something (a count, a total, a list). Neither call reads the journal, so their cost does not grow with the history.

Retrieve. Each search ranks the journal’s records by BM25 and by embedding similarity, fuses the two rankings by RRF [8], and returns a page of records. The pages merge into a *pool*: each search first contributes an equal quota of its top results, and the remaining slots go to the highest summed reciprocal rank.

Judge (System 1). Judging is System 1 work: each proposition concerns one record, has an explicit criterion and can be answered independently of the others. Jev answers typed yes/no propositions and returns the probability of yes [1]. It *screens* each pooled record (should the reply use this record?), then *judges* the records that could enter the View, jointly and with the last six messages, as *needed* or *no longer current* (a later message or record explicitly changes, cancels or completes it), and fills a *need table*: does this record satisfy this need? Jev then judges which index items the reply needs, among those linked to the judged records and the dozen nearest to the message. Counting, arithmetic, comparing dates and multi-hop reasoning are System 2 work, which Jev handles poorly [46]; they are

Algorithm 1 One user turn of Mnemon.

Input: the last six messages M , the journal J and its index X

```
1:  $(Q, N) \leftarrow \text{PLAN}(M)$  ▷ System 2: five searches, at most three needs
2:  $P \leftarrow \text{POOL}(\{\text{SEARCH}(J, q) : q \in Q\})$  ▷ BM25 and embeddings, fused by RRF; at most 48 records
3: for round = 1, 2, 3 do
4:    $\text{SCREEN}(P); \text{JUDGE}(P, M, N)$  ▷ System 1: use it? needed? no longer current? which need?
5:    $A \leftarrow \text{ACT}(N, P)$  ▷ rules: page a search, ask System 2 for a new one, or drop the need
6:   if  $A = \emptyset$  then
7:     break
8:    $P \leftarrow P \cup \text{RUN}(A)$ 
9:  $I \leftarrow \text{JUDGEITEMS}(X, P, M)$  ▷ index items linked to judged records, and the nearest
10: return  $\text{COMPOSE}(I, P)$  ▷ index first, then records, within the View's budgets
```

left to the planner and the answering model.

4.3 Rules and Budgets

The rules connect the two systems. They read only what System 1 reliably gives, its yes/no decision ($p \geq 1/2$) and its ranking, together with events observed in the previous round, and they call on System 2 only when System 1 reports trouble: a need that no judged record satisfies prompts the planner for a new search, much as System 2 is mobilized when System 1 runs into difficulty [18]. Every other constant is an interpretable budget: three needs per turn, pages of 20 records or 12,000 characters, a pool of 48 records, three rounds, and a View of 16 records and 12,000 characters. Because the rules use judgments only through their order and their decisions, they do not depend on how the decision model is calibrated.

Proposition 1 (Calibration invariance). *Let h be a strictly increasing map of $[0, 1]$ onto itself with $h(1/2) = 1/2$. Replacing every judgment p by $h(p)$ leaves every action of the loop, and hence the View, unchanged.*

Proof. The rules compare judgments only with each other or with $1/2$, and h preserves both comparisons. □

A decision model can thus be replaced by another that orders and decides alike, without retuning any rule.

The loop. Each open need gets one action per round. A single need is met once a record satisfies it. A need for every instance, labeled ALL or observed because two records satisfy it, pages each search that found a new positive until none does. A need that no record satisfies yet follows its *lead*, the record the need table ranks highest: the loop pages the search that ranked the lead highest, then asks the planner for one new search, then drops the need.

The View. The View leads with the index, each block within a fixed budget: standing instructions, a one-line directory of topics, and the timelines and value histories Jev judged needed. Records follow: those judged needed, then, when the planner named a need, the rest in Jev's ranking up to the View's budget, trimmed from the end to make room for the index. Records judged no longer current are left out.

Work per turn. With these budgets, the work on a turn's critical path does not grow with the history, except in search. A turn that makes R sequential System 2 calls (the plan, any new search the loop asks for, and the answer), W waves of judgments and S reads of the journal takes

$$T = \sum_{i=1}^R \tau_i^{(2)} + \sum_{w=1}^W \tau_w^{(1)} + \sum_{s=1}^S \tau_s^{\text{read}}(n), \quad (3)$$

where $\tau_i^{(2)}$, $\tau_w^{(1)}$ and $\tau_s^{\text{read}}(n)$ are the latencies of the i th System 2 call, the w th wave and the s th read, each read being over the n records of the journal. The rounds and the needs bound R , W and S , and only reads depend on n ; since reads spend no model tokens, the tokens a turn spends are bounded as well.

5 Implementation

Mnemon runs as a *replica*: a second DSH instance with the same Sources, its own Strategy and a control loop that calls Jev. At each user turn the replica runs one job and publishes its View into the main instance. Upstream DSH is unmodified, and apart from 115 changed lines of `dsh-mnemon` [50] the replica consists of new plugins, released with this paper [49].

Journal. Records are chunks of up to about 600 characters, embedded locally with `nomic-embed-text` [37]; a LongMemEval-S history of about 106k tokens becomes roughly 1,000 records, written in under a second. For BEAM-10M, the journal is sized for up to 200,000 records per conversation and keeps its search index between searches.

Consolidation. Background jobs read the journal from a watermark, in batches of about 10,000 characters that hold user messages whole and the first 240 characters of each assistant message, about 15% of a conversation’s text. DeepSeek-V4.1-Flash without thinking [10] folds each batch into the index. For very long histories a fold sees at most 120 topics, 240 values and 60 instructions, half chosen by the words they share with the batch and half by recency. Index items are embedded as they are written, so that no View waits for them.

Models. System 2 is an LLM: the planner and the answering model are the same LLM, `gpt-4.1-mini` or DeepSeek-V4.1-Flash (Section 6.1). System 1 is Jev 1.13, which judges up to 40 records per call and answers every proposition about them in that call (Section 6.5).

6 Evaluation

We evaluate Mnemon on four public benchmarks and ask six questions. How accurate is it at small context, compared with systems re-evaluated under one protocol (Section 6.2)? What does a stronger System 2 add, and how does the result compare with the best published results (Section 6.3)? Does its cost stay bounded as the history grows to 10M tokens (Section 6.4)? Is judging better done by System 1 than by an LLM (Section 6.5)? Is the same System 1 better used to judge raw records at read time than to organize memory at write time (Section 6.6)? And what does consolidation add (Section 6.7)?

6.1 Setup

Benchmarks. LoCoMo [29] has 10 conversations of 13–25k tokens and 1,540 non-adversarial questions. LongMemEval-S [54, 55] has 500 questions, each with its own history of about 106k tokens. HaluMem [5] (Medium set) has 20 users and 3,467 questions. BEAM [44] has two tiers: BEAM-100K, with 20 conversations and 400 questions over 10 abilities, and BEAM-10M, with 10 conversations of about 90,000 records each and 200 questions. LoCoMo is reported with its original labels; for comparison with published claims we also give our score under our revised labels, which drop 44 unusable questions and correct 25 gold answers.

Models. In the *standard setting*, `gpt-4.1-mini` [38] (snapshot 2025-04-14, temperature 0) answers and plans, as in OmniMemEval [33]. In the *reasoning setting*, DeepSeek-V4.1-Flash [10] answers with thinking and plans without.

Grading. LoCoMo answers are graded with the lenient prompt of Mem0 [7] and later work, LongMemEval-S answers with its per-type prompts, and HaluMem and BEAM answers with their official prompts and rubrics. The primary grader is `gpt-4.1-mini` in the standard setting and DeepSeek-V4.1-Flash (without thinking, temperature 0) in the reasoning setting, and the other grades every answer as well; on LoCoMo and LongMemEval-S the two agree on 96–97% of answers (Cohen’s κ 0.85–0.89). OmniMemEval grades with `gpt-4o-mini`, so comparisons with it carry a grader difference of 1–2 points. Paired differences between configurations are reported with 95% confidence intervals (CIs).

Baselines. We compare Mnemon with the 14 systems that OmniMemEval re-evaluated with `gpt-4.1-mini` answering, using its published numbers, and, separately, with the best result each open-source project has published, whatever its model and protocol.

Cost accounting. Costs follow the token counts the APIs return, at list prices per million tokens (gpt-4.1-mini \$0.40 input, \$0.10 cached, \$1.60 output; DeepSeek-V4.1-Flash off-peak \$0.15, \$0.003, \$0.60; Jev \$0.042 input), and cover the answer, the planner and Jev; consolidation is a one-time cost per history. Questions that failed on a transient error or received an empty View were answered once more.

6.2 Accuracy at Small Context

Table 1 places Mnemon among the 14 systems that OmniMemEval re-evaluated with the same answering model. On LoCoMo it scores 91.7%, against 88.83% for MemOS and at most 83.48% for the rest, a margin beyond the 1–2 points that separate graders; on LongMemEval-S it scores 83.8%, second to MemOS (89.2%) and ahead of EverMemOS (80.4%) and Zep (79.8%). It sends the answering model about 3.8k tokens per question, less than any other system above 80% on either benchmark (MemOS 4.2–5.4k, EverMemOS 8.6–12.4k, Hindsight 24.7k, Cognee 32.5k). The context is small because System 1 does the broad reading: per question, Jev reads 35–73k tokens of records, 9–19 times what the answering model reads, at about a tenth of its price per token, and passes on only what the reply needs. With c_{full} set to 21,613 gpt-4.1-mini input tokens on LoCoMo and 105,134 on LongMemEval-S, Mnemon has the lowest effective cost index (Equation 2) of the 15 systems on LoCoMo, 0.259 against 0.337 for mem9, and the second lowest on LongMemEval-S, 0.198 after MemOS at 0.147 (Figure 1).

Table 1: Mnemon and the 14 systems re-evaluated by OmniMemEval [33], all with gpt-4.1-mini answering: accuracy, context sent to the answering model per question, and effective cost index (ECI, Equation 2; lower is better). OmniMemEval grades with gpt-4o-mini and we with gpt-4.1-mini; with DeepSeek as grader, Mnemon scores 91.4% and 85.4%. Best per column in bold.

system	LoCoMo (1,540 questions)			LongMemEval-S (500 questions)		
	accuracy (%)	context (k tokens)	ECI	accuracy (%)	context (k tokens)	ECI
Mnemon	91.7	3.8	0.259	83.8	3.8	0.198
MemOS	88.83	5.4	0.362	89.2	4.2	0.147
Cognee	83.48	32.5	1.670	51.8	10.3	0.580
EverMemOS	82.75	8.6	0.569	80.4	12.4	0.314
Hindsight	81.99	24.7	1.322	72.2	29.8	0.561
Mem0	77.68	17.4	1.028	56.0	0.9	0.448
Letta	77.12	14.2	0.885	77.67	49.4	0.693
MemMachine	73.9	2.6	0.380	63.6	2.8	0.391
mem9	73.64	1.6	0.337	78.0	3.8	0.256
Supermemory	73.53	15.2	0.970	66.07	6.6	0.402
MemoryLake	72.49	5.2	0.516	–	–	–
Viking Memory	69.33	6.0	0.583	61.07	2.3	0.411
Zep	63.83	1.9	0.448	79.8	117.1	1.316
Memori	41.34	8.1	0.963	20.8	2.8	0.818
Backboard.io	22.4	1.2	0.831	–	–	–

By question type (Table 2), Mnemon leads on every LoCoMo type except open-domain, whose questions ask for inferences the conversation does not state, with the largest margins on temporal and multi-hop questions. On LongMemEval-S it leads on knowledge updates, matches MemOS on multi-session questions and trails it mainly on temporal-reasoning and preference questions, which require computing or inferring at answer time.

6.3 A Stronger System 2

System 2 can be replaced without changing the memory. With DeepSeek-V4.1-Flash planning and answering, with thinking when it answers, Mnemon scores 92.2% on LoCoMo and 94.4% on LongMemEval-S under the DeepSeek grader (92.8% and 93.4% under gpt-4.1-mini), at \$2.38 and \$2.53 per 1,000 questions. On LongMemEval-S the stronger System 2 adds 9.6 points on the same memory under the gpt-4.1-mini grader (95% CI +6.3 to +12.9), most on temporal-reasoning and multi-session questions, whose computing and counting are System 2 work; with it, Mnemon reaches

Table 2: Accuracy (%) by question type. Mnemon is graded by gpt-4.1-mini in the standard setting (gpt-4.1-mini as System 2) and the reasoning setting (DeepSeek-V4.1-Flash); MemOS and EverMemOS, the strongest systems OmniMemEval re-evaluated, as it reports them (gpt-4.1-mini answering, gpt-4o-mini grading). Bold: best of the three systems with gpt-4.1-mini answering.

question type	questions	gpt-4.1-mini answering			reasoning
		Mnemon	MemOS	EverMemOS	Mnemon
<i>LoCoMo</i>					
single-hop	841	94.6	92.51	86.8	96.1
multi-hop	282	91.8	88.65	77.78	92.2
temporal	321	91.3	85.05	84.11	92.2
open-domain	96	66.7	69.79	57.29	67.7
<i>LongMemEval-S</i>					
single-session, user	70	95.7	100.0	91.43	97.1
single-session, assistant	56	94.6	100.0	89.29	100.0
single-session, preference	30	76.7	100.0	96.67	100.0
temporal reasoning	133	76.7	89.47	81.95	92.5
multi-session	133	78.2	78.95	66.17	88.0
knowledge update	78	89.7	84.62	79.49	93.6

or passes MemOS on five of the six LongMemEval-S question types (Table 2). Raw records leave computing and counting to the answering model, and a stronger one does them. Table 3 lists each open-source project’s best published result, whatever its answering model and protocol. On LongMemEval-S Mnemon is on par with the best claims (Hindsight 94.6%, Mem0 94.4% with gpt-5); on LoCoMo, where claims reach 94.7% (Zep with gpt-5.4), it scores 92.2% on the original labels and 95.3% on the revised ones.

Table 3: Each open-source project’s best published result, with whatever answering model, grader and protocol it used (checked September 26–29, 2026). The settings differ widely, so the table ranks claims, not systems. [†]On the revised LoCoMo labels (Section 6.1); 92.2 on the original labels, which every other entry uses. [‡]Including the adversarial category.

project	LoCoMo	LongMemEval-S	answering model / grader	method
Mnemon	95.3 [†]	94.4	DeepSeek-V4.1-Flash (thinking) / DeepSeek	raw records and a consolidated index, LLM-planned search, Jev judging
Zep / Graphiti [60]	94.7	90.2	gpt-5.4 (medium reasoning) / gpt-5.4	temporal knowledge graph + reranking
EverMemOS [16]	93.05	83.0	gpt-4.1-mini / three graders averaged	MemCell to MemScene
Mem0 [30]	92.5	94.4	gpt-5 / gpt-5	LLM-extracted facts, top 200 (about 7k tokens)
memU [35]	92.09	–	not stated (early version)	–
Hindsight [21, 47]	92.0	94.6	not stated (paper: gemini-3-pro 89.6 / 91.4)	four memory networks, 36–44k tokens of context
MemMachine [52]	91.69	93.0	gpt-4.1-mini; LongMemEval-S gpt-5-mini / gpt-4o-mini	raw episodes indexed by sentence
MemOS [33]	88.83	89.2	gpt-4.1-mini / gpt-4o-mini (OmniMemEval)	MemCube tree / graph memory
Memori [4]	87.0	–	gpt-4.1-mini / gpt-4.1-mini	LLM-extracted triples + summaries
mem9 [31]	86.85	–	qwen3.6-plus / qwen3.6-plus	keyword + vector retrieval
MIRIX [53]	85.38	–	gpt-4.1-mini / gpt-4.1	multi-agent, six memory types
Nemori [34]	83.05	74.6	gpt-4.1-mini / not stated	episodic narratives + distilled facts
OpenViking [48]	82.86	–	Doubao 2.0 Pro / Doubao	file-system context store, three tiers
Jev-Mem [17]	77.7 [‡]	–	gpt-4o-mini / not stated	Jev-typed turns in a relation graph built at write time
Memobase [32]	75.78	–	not stated	user profile + event timeline
Letta [22]	74.0	–	gpt-4o-mini / gpt-4.1	agent searches raw conversation files
LightMem [13]	72.99	73.2	gpt-4o-mini; LongMemEval-S glm-4.6 / not stated	pre-compression + offline update
Supermemory [43]	–	85.2	gemini-3-pro / not stated	–
SimpleMem [28]	–	84.4	gpt-4.1 / gpt-4.1-mini	atomic memory units
Engram [51]	–	83.6	doubao-seed-2.0-pro / deepseek-v3.2	bi-temporal knowledge graph from asynchronous extraction

6.4 Other Benchmarks and Scale

Table 4 reports Mnemon on every benchmark and tier with its full cost per question. HaluMem and BEAM test what the design targeted least: HaluMem’s grader counts an answer that adds details beyond the reference as a

hallucination, and BEAM asks for instructions to be followed, contradictions to be reported and whole conversations to be summarized. On them, Mnemon ranks eighth of 13 systems on HaluMem and tenth of 12 on BEAM-100K.

Table 4: Mnemon on each benchmark and tier with gpt-4.1-mini answering: score under each grader (accuracy; HaluMem: share correct; BEAM: rubric score), rank among the systems OmniMemEval re-evaluated, context and cost per question, one-time consolidation cost per history, and the median question’s Jev calls (sequential waves) and searches, with the warm latency of one search on the largest history.

benchmark	score, grader		rank	context (k tokens)	cost (\$)		median question		
	gpt-4.1-mini	DeepSeek			/1k q.	/history	Jev calls (waves)	searches	search (ms)
LoCoMo	91.7	91.4	1/15	3.8	3.27	0.013	5 (4)	7	14
LongMemEval-S	83.8	85.4	2/13	3.8	3.33	0.017	5 (5)	7	14
HaluMem	73.3	65.8	8/13	3.4	3.60	0.089	7 (6)	10	18
BEAM-100K	64.5	60.5	10/12	3.8	4.80	0.012	9 (7)	13	17
BEAM-10M	51.2	48.8	10/12	3.8	5.32	1.62	10 (7)	14	248

Nothing on the read path of Mnemon grows with the history except the search index: the planner reads the recent dialogue, Jev screens at most 48 records a round, and the View has fixed budgets. BEAM-10M tests this with conversations of about 90,000 records, 80 times as many as BEAM-100K’s. From BEAM-100K to BEAM-10M the cost per question grows by a factor of 1.11, and the work on a question’s critical path stays about the same. On BEAM-10M Mnemon scores 51.2, against 43.3–59.8 for the 11 systems OmniMemEval reports, and consolidating its 10 conversations took 6,095 folds without a failure.

We report latency as critical-path work (Equation 3, Table 4). For the median question, System 2 makes two sequential calls, the plan (two requests in parallel) and the answer, and a third in the 6–31% of questions whose loop asks for a new search; System 1 makes 5–10 calls in 4–7 waves, 1.4–2.4 s at 0.34 s a wave (Figure 3b). A warm search takes 14–18 ms, mostly to embed the query, and a question’s reads 0.1–0.2 s, but 248 ms and 3.5 s on BEAM-10M’s largest history (108,810 records), because our search scores every record; inverted and approximate nearest-neighbor indexes would avoid this.

6.5 System 1 Against LLMs

The division gives judging to System 1. To test it, we gave gpt-4.1-mini and DeepSeek the proposition Jev answers, with the same dialogue and records, for 599 LoCoMo and LongMemEval-S questions (14,359 records, 1,004 of them gold evidence). Jev separates gold evidence best (Figure 3): AUC 0.942 overall against 0.900 for DeepSeek and 0.853 for gpt-4.1-mini, and 0.939 against 0.872 and 0.798 on LongMemEval-S, whose chat-turn records rarely repeat the question’s words. A Jev call on 24 records takes 0.34 s while answering two propositions per record; the LLMs take 1.05–3.74 s for one (Figure 3b). System 1 work is thus done better, and 3–11 times faster, by a decision model: over the 4–7 waves of a question (Table 4), judging with an LLM would add seconds and cost while separating evidence less well.

6.6 Judging Raw Records or Organizing Them

The design can also be tested against a system that uses the same System 1 at write time. Jev-Mem [17], concurrent work, gives Jev the decisions of memory management: when a turn is written, Jev types it and judges its relations to earlier turns, building a graph with semantic, temporal, causal and entity edges; when a question arrives, Jev routes it over the graph, allocates the search budget and decides when to stop, and an LLM writes the answer. We ran its released code (default profile, Jev 1.13 as in our runs) under our protocol: the same 1,540 LoCoMo questions, gpt-4.1-mini answering once per question from the question alone, and both graders. Its released runner, by default, chooses among three answers by comparing them with the gold answer and picks its answer prompt by the question’s category; we did neither.

Jev-Mem scores 84.4% (82.1% under DeepSeek), 7.3 points below Mnemon (95% CI +5.5 to +9.2; 166 questions answered correctly only by Mnemon, 53 only by Jev-Mem), with the largest gaps on multi-hop (14.2 points) and temporal (8.4) questions, which need records connected or dated; given each question’s category, it scores 84.0%. It sends less context (2.6k tokens; ECI 0.275 against 0.259) and does less work per question, one LLM call and 4.1 Jev

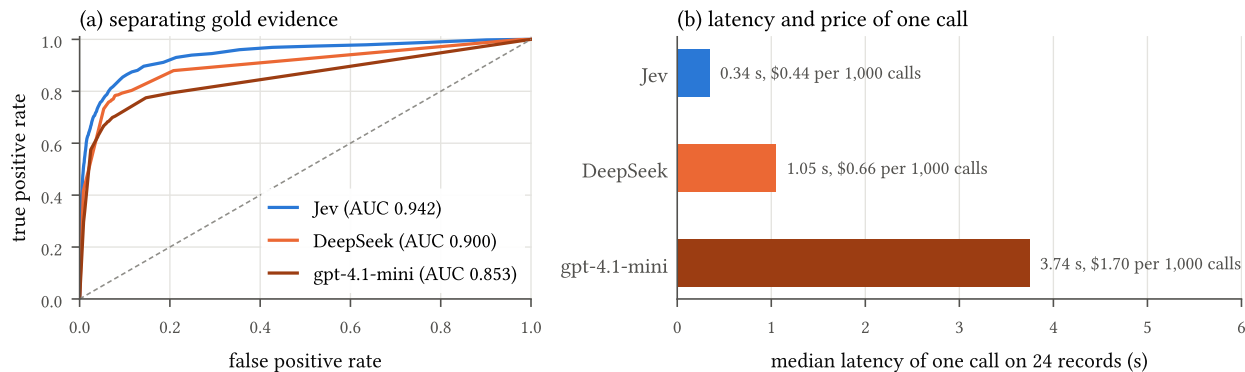


Figure 3: Jev and two LLMs on the same 14,359 records: (a) ROC curves for separating gold evidence, pooled over records; (b) median latency and list price of one call on 24 records, in which Jev also answers whether each is no longer current.

calls on average, but it spends 9.5 times as much per history at write time (\$0.12 against \$0.013), where Jev judges every turn against its candidates. The two systems differ in more than where System 1 works, so this is not an ablation; but with the decision model held fixed, judging raw records once the question is known was more accurate than organizing them before it was.

6.7 What Consolidation Adds

Consolidation is the System 2 work that Mnemon moves off the read path, and answering the same questions with the index removed measures what it adds. With gpt-4.1-mini answering it adds 2.5 points on LoCoMo, 4.4 on LongMemEval-S, 6.2 on BEAM-100K and 5.9 on BEAM-10M, and 1.9 on LoCoMo and 4.0 on LongMemEval-S with the reasoning model as System 2, each with a 95% CI above zero; on HaluMem the two graders disagree in sign. The gains fall where a question does not point to its evidence: instruction following, knowledge updates, summarization and abstention on BEAM; knowledge updates and multi-session questions on LongMemEval-S; and multi-hop and temporal questions on LoCoMo, whose facts the timelines connect and date. Consolidation multiplies the cost per question by 1.17–1.37, mostly for Jev to judge the index items.

7 Limitations

Development and evaluation. Mnemon was developed on LoCoMo and LongMemEval-S, and consolidation was designed after we had seen results on HaluMem and BEAM-100K, so none of the benchmarks is held out. Our runs are graded by gpt-4.1-mini and DeepSeek and OmniMemEval’s by gpt-4o-mini; graders differ by 1–2 points, and on HaluMem by more. Each configuration ran once per setting, and two identical runs of an earlier configuration differed by up to 2.6 points. We evaluate only conversational memory held in one Source; stores of documents, tasks or logs, which the design admits, are untested.

Dependence on Jev. Mnemon relies on Jev, a hosted, proprietary model; LLMs can stand in with lower discrimination at several times the latency and price, but no substitute was tested in the full system.

Latency and privacy. Our runs shared one laptop and public model APIs; a question took 10–15 s end to end at the median, mostly in model calls. Raw records keep everything an extractor would discard, and the index repeats some of it, so deletion and retention policies must cover both.

8 Conclusion

Mnemon divides the work of memory between a slow System 2, an LLM that writes a few search queries, names what a reply needs and composes the answer, and a fast System 1, a decision model that makes many small, explicit judgments about the records those searches find; slow work that no reply can wait for, consolidating each record

once, runs in the background. Because nothing about a record is decided when it is written, the same agent can read any store that returns dated records, a general way to give agents a past beyond conversation. With a small answering model, Mnemon is the most accurate of the systems in a public re-evaluation on LoCoMo and second on LongMemEval-S, from under 4k tokens of context per question and with the lowest effective cost index on LoCoMo; with a reasoning model as System 2 it is on par with the best published results on LongMemEval-S; and its cost per question stays nearly flat from 100K to 10M tokens of history. Most of the read-time work of memory is System 1 work, which a decision model does better, faster and more cheaply than a general LLM; judging raw records once the question is known was also more accurate than using the same model to organize them in advance, and raw records leave room for stronger answering models. Code, prompts, analysis scripts and run records (HaluMem’s excepted) are available in the Mnemon repository [49].

References

- [1] Diogo Almeida. Introducing System One models and Jev. TypeSafe AI blog, September 2026. URL <https://typesafe.ai/blog/introducing-system-one-models-and-jev>. Model documentation and prices at <https://docs.typesafe.ai/models>; accessed 2026-09-26.
- [2] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In *International Conference on Learning Representations (ICLR)*, pages 9112–9141, 2024.
- [3] Grady Booch, Francesco Fabiano, Lior Horesh, Kiran Kate, Jon Lenchner, Nick Linck, Andrea Loreggia, Keerthiram Murugesan, Nicholas Mattei, Francesca Rossi, and Biplav Srivastava. Thinking fast and slow in AI. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, pages 15042–15046, 2021.
- [4] Luiz C. Borro, Luiz A. B. Macarini, Gordon Tindall, Michael Montero, and Adam B. Struck. Memori: A persistent memory layer for efficient, context-aware LLM agents. arXiv preprint arXiv:2603.19935, 2026.
- [5] Ding Chen, Simin Niu, Kehang Li, Peng Liu, Xiangping Zheng, Bo Tang, Xinchu Li, Feiyu Xiong, and Zhiyu Li. HaluMem: Evaluating hallucinations in memory systems of agents. arXiv preprint arXiv:2511.03506, 2025.
- [6] Lingjiao Chen, Matei Zaharia, and James Zou. FrugalGPT: How to use large language models while reducing cost and improving performance. *Transactions on Machine Learning Research*, 2024.
- [7] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready AI agents with scalable long-term memory. In *ECAI 2025: 28th European Conference on Artificial Intelligence*, volume 413 of *Frontiers in Artificial Intelligence and Applications*, pages 2993–3000. IOS Press, 2025.
- [8] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Buettcher. Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 758–759, Boston, MA, USA, 2009. ACM.
- [9] Thomas M. Cover and Joy A. Thomas. *Elements of Information Theory*. Wiley-Interscience, Hoboken, NJ, second edition, 2006.
- [10] DeepSeek-AI. DeepSeek-V4.1-Flash: Smarter, faster, more efficient. DeepSeek API news, September 2026. URL <https://api-docs.deepseek.com/news/news260910>. Prices at https://api-docs.deepseek.com/quick_start/pricing/; accessed 2026-09-26.
- [11] DeepSeek-AI. DeepSeek Harness: Everything is a plugin. GitHub repository, 2026. URL <https://github.com/deepseek-ai/deepseek-harness>. Accessed 2026-09-26.
- [12] Jesper D erhag, Carlos Calva, and Timmy Ghiurau. SmartSearch: How ranking beats structure for conversational memory retrieval. arXiv preprint arXiv:2603.15599, 2026.
- [13] Jizhan Fang, Xinle Deng, Haoming Xu, Ziyang Jiang, Yuqi Tang, Ziwen Xu, Shumin Deng, Yunzhi Yao, Mengru Wang, Shuofei Qiao, Huajun Chen, and Ningyu Zhang. LightMem: Lightweight and efficient memory-augmented generation. In *International Conference on Learning Representations (ICLR)*, pages 98706–98729, 2026.

- [14] Tianxiang Fei, Mingyang Song, Mao Zheng, and Xiang Yu. Memory beyond recall: A dual-process cognitive memory system for self-evolving LLM agents. arXiv preprint arXiv:2606.09483, 2026.
- [15] Luyu Gao, Xueguang Ma, Jimmy Lin, and Jamie Callan. Precise zero-shot dense retrieval without relevance labels. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1762–1777, Toronto, Canada, July 2023. Association for Computational Linguistics.
- [16] Chuanrui Hu, Xingze Gao, Zuyi Zhou, Dannong Xu, Yi Bai, Xintong Li, Hui Zhang, Tong Li, Chong Zhang, Lidong Bing, and Yafeng Deng. EverMemOS: A self-organizing memory operating system for structured long-horizon reasoning. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 45836–45853, San Diego, California, United States, July 2026. Association for Computational Linguistics.
- [17] Dongming Jiang, Yi Li, and Bingzhe Li. Jev-Mem: System-one-controlled agentic memory for efficient AI agents. arXiv preprint arXiv:2609.23986, 2026. Code at <https://github.com/libingzheren/Jev-Mem>.
- [18] Daniel Kahneman. *Thinking, Fast and Slow*. Farrar, Straus and Giroux, New York, 2011.
- [19] Jiazheng Kang, Mingming Ji, Zhe Zhao, and Ting Bai. Memory OS of AI agent. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 25961–25970, Suzhou, China, November 2025. Association for Computational Linguistics.
- [20] Sangyeop Kim, Yohan Lee, Sanghwa Kim, Hyunjong Kim, and Sungzoon Cho. Pre-storage reasoning for episodic memory: Shifting inference burden to memory for personalized dialogue. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 22096–22113, Suzhou, China, November 2025. Association for Computational Linguistics.
- [21] Chris Latimer, Nicolò Boschi, Andrew Neeser, Chris Bartholomew, Gaurav Srivastava, Xuan Wang, and Naren Ramakrishnan. Hindsight is 20/20: Building agent memory that retains, recalls, and reflects. arXiv preprint arXiv:2512.12818, 2025.
- [22] Letta. Letta (formerly MemGPT): Platform for stateful agents. GitHub repository, 2024. URL <https://github.com/letta-ai/letta>. LoCoMo results at <https://www.letta.com/blog/benchmarking-ai-agent-memory/>; accessed 2026-09-26.
- [23] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020.
- [24] Dongfang Li, Zixuan Liu, Junmai Wang, Jiahe Huang, Fuhao Li, Bonian Jia, Baotian Hu, and Min Zhang. LycheeMemory V2: Efficient long-term memory for LLM agents via semantic segment-level consolidation. arXiv preprint arXiv:2608.12990, 2026.
- [25] Zhiyu Li, Chenyang Xi, Chunyu Li, Ding Chen, Boyu Chen, Shichao Song, Simin Niu, Hanyu Wang, Jiawei Yang, Chen Tang, et al. MemOS: A memory OS for AI system. arXiv preprint arXiv:2507.03724, 2025.
- [26] Yuxin Liao, Le Wu, Min Hou, Hao Liu, Han Wu, and Zishu Wang. LeanMem: Simple and efficient long-term memory for LLM agents. arXiv preprint arXiv:2608.03463, 2026.
- [27] Bill Yuchen Lin, Yicheng Fu, Karina Yang, Faeze Brahman, Shiyu Huang, Chandra Bhagavatula, Prithviraj Ammanabrolu, Yejin Choi, and Xiang Ren. SwiftSage: A generative agent with fast and slow thinking for complex interactive tasks. In *Advances in Neural Information Processing Systems*, volume 36, 2023.
- [28] Jiaqi Liu, Yaofeng Su, Peng Xia, Siwei Han, Zeyu Zheng, Cihang Xie, Mingyu Ding, and Huaxiu Yao. SimpleMem: Efficient lifelong memory for LLM agents. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, 2026.

- [29] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of LLM agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 13851–13870, Bangkok, Thailand, August 2024. Association for Computational Linguistics.
- [30] Mem0. Mem0: The memory layer for AI agents. GitHub repository, 2026. URL <https://github.com/mem0ai/mem0>. Results at <https://mem0.ai/research>; accessed 2026-09-26.
- [31] mem9.ai. mem9: Persistent memory for AI agents. GitHub repository, 2026. URL <https://github.com/mem9-ai/mem9>. LoCoMo baseline in benchmark/BASELINE.md; accessed 2026-09-26.
- [32] Memobase. Memobase: User profile-based long-term memory for AI chatbot applications. GitHub repository, 2025. URL <https://github.com/memodb-io/memobase>. LoCoMo results in docs/experiments/locomo-benchmark; accessed 2026-09-26.
- [33] MemTensor. OmniMemEval: Evaluation framework for benchmarking memory systems. GitHub repository, 2026. URL <https://github.com/MemTensor/OmniMemEval>. User-memory results for 14 systems at https://github.com/MemTensor/OmniMemEval/blob/main/docs/user_memory/results.md; accessed 2026-09-26.
- [34] Jiayan Nan, Wenquan Ma, Wenlong Wu, and Yize Chen. Nemori: Self-organizing agent memory inspired by cognitive science. arXiv preprint arXiv:2508.03341v3, 2025.
- [35] NevaMind AI. memU: Memory for proactive AI agents. GitHub repository, 2026. URL <https://github.com/NevaMind-AI/memU>. LoCoMo result at <https://memu.pro/benchmark>; accessed 2026-09-26.
- [36] Rodrigo Nogueira and Kyunghyun Cho. Passage re-ranking with BERT. arXiv preprint arXiv:1901.04085, 2019.
- [37] Zach Nussbaum, John X. Morris, Brandon Duderstadt, and Andriy Mulyar. Nomic Embed: Training a reproducible long context text embedder. *Transactions on Machine Learning Research*, 2025.
- [38] OpenAI. Introducing GPT-4.1 in the API. OpenAI blog, April 2025. URL <https://openai.com/index/gpt-4-1/>. Model page and prices at <https://developers.openai.com/api/docs/models/gpt-4.1-mini>; accessed 2026-09-26.
- [39] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. arXiv preprint arXiv:2310.08560, 2023.
- [40] Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. Zep: A temporal knowledge graph architecture for agent memory. arXiv preprint arXiv:2501.13956, 2025.
- [41] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.
- [42] Weiwei Sun, Lingyong Yan, Xinyu Ma, Shuaiqiang Wang, Pengjie Ren, Zhumin Chen, Dawei Yin, and Zhaochun Ren. Is ChatGPT good at search? Investigating large language models as re-ranking agents. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 14918–14937, Singapore, December 2023. Association for Computational Linguistics.
- [43] Supermemory. Supermemory: Memory and context engine for AI agents. GitHub repository, 2026. URL <https://github.com/supermemoryai/supermemory>. Results at <https://supermemory.ai/research>; accessed 2026-09-26.
- [44] Mohammad Tavakoli, Alireza Salemi, Carrie Ye, Mohamed Abdalla, Hamed Zamani, and J. Ross Mitchell. Beyond a million tokens: Benchmarking and enhancing long-term memory in LLMs. In *International Conference on Learning Representations (ICLR)*, pages 133343–133429, 2026.
- [45] Anxin Tian, Yiming Li, Xing Li, Hui-Ling Zhen, Lei Chen, Xianzhi Yu, Zhenhua Dong, and Mingxuan Yuan. SwiftMem: Fast agentic memory via query-aware indexing. arXiv preprint arXiv:2601.08160, 2026.
- [46] TypeSafe AI. Model jaggedness: Jev 1.13. Model documentation, 2026. URL <https://docs.typesafe.ai/model-jaggedness/jev-1.13>. Accessed 2026-09-26.

- [47] Vectorize. Hindsight: Agent memory that learns over time. GitHub repository, 2026. URL <https://github.com/vectorize-io/hindsight>. Results at <https://benchmarks.hindsight.vectorize.io/>; accessed 2026-09-26.
- [48] Volcengine. OpenViking: Self-evolving context database for AI agents. GitHub repository, 2026. URL <https://github.com/volcengine/OpenViking>. Benchmark results at <https://blog.openviking.ai/post/openviking-benchmark-results/>; accessed 2026-09-26.
- [49] Guangren Wang. mnemon-memory-agent: Code, prompts, run records and analysis scripts of Mnemon. GitHub repository, 2026. URL <https://github.com/Grivn/mnemon-memory-agent>. Code under the MIT license; run records under each benchmark’s license, HaluMem’s withheld.
- [50] Guangren Wang. dsh-mnemon: Composable memory for DeepSeek Harness. GitHub repository, 2026. URL <https://github.com/omdsh-dev/dsh-mnemon>. MIT license.
- [51] Liuyin Wang. Less context, more accuracy: A bi-temporal memory engine for LLM agents where a lean retrieved context beats the full history. arXiv preprint arXiv:2606.09900, 2026. Code at <https://github.com/ly-wang19/engram>.
- [52] Shu Wang, Edwin Yu, Oscar Love, Tom Zhang, Tom Wong, Steve Scargall, and Charles Fan. MemMachine: A ground-truth-preserving memory system for personalized AI agents. arXiv preprint arXiv:2604.04853, 2026.
- [53] Yu Wang and Xi Chen. MIRIX: Multi-agent memory system for LLM-based agents. arXiv preprint arXiv:2507.07957, 2025.
- [54] Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. LongMemEval cleaned release. Hugging Face dataset, September 2025. URL <https://huggingface.co/datasets/xiaowu0162/longmemeval-cleaned>. Accessed 2026-09-26.
- [55] Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. LongMemEval: Benchmarking chat assistants on long-term interactive memory. In *International Conference on Learning Representations (ICLR)*, pages 86809–86836, 2025.
- [56] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-Mem: Agentic memory for LLM agents. In *Advances in Neural Information Processing Systems*, volume 38, pages 17577–17604, 2025.
- [57] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
- [58] Zhixing You, Jiachen Yuan, and Jason Cai. D-Mem: A dual-process memory system for LLM agents. arXiv preprint arXiv:2603.18631, 2026.
- [59] Jiawei Yu, Yixiang Fang, Xilin Liu, and Yuchi Ma. H-Mem: A novel memory mechanism for evolving and retrieving agent memory via a hybrid structure. arXiv preprint arXiv:2605.15701, 2026.
- [60] Zep. Graphiti: A framework for building temporal knowledge graphs. GitHub repository, 2026. URL <https://github.com/getzep/graphiti>. Results at <https://www.getzep.com/research/>; accessed 2026-09-26.
- [61] Xiaochen Zhao, Kaikai Wang, Xiaowen Zhang, Chen Yao, and Aili Wang. HyMem: Hybrid memory architecture with dynamic retrieval scheduling. arXiv preprint arXiv:2602.13933, 2026.
- [62] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems (Datasets and Benchmarks Track)*, volume 36, pages 46595–46623, 2023.
- [63] Sizhe Zhou and Jiawei Han. A simple yet strong baseline for long-term conversational memory of LLM agents. arXiv preprint arXiv:2511.17208, 2025.